코드 설명 보고서

Q-Learning을 활용한 FrozenLake 강화학습 구현 및 학습 성능 분석

학번 : 2026406076

이름 : 정창규

제출일 : 2026-10-30

목차

1. 과제 개요 및 목적

2. FrozenLake Q-Learning 구현

- 2.1 FrozenLake 환경 및 Q-table 구성
- 2.2 ε-greedy 행동 선택
- 2.3 Q-Learning 학습 및 평가 과정
- 2.4 학습 결과 시각화

3. 실행 결과 및 분석

- 3.1 학습 성공률 및 평균 보상
- 3.2 Q-table 히트맵 및 정책 격자
- 3.3 강화학습 핵심 개념 및 Q값 갱신 과정

4. 결론 및 고찰

5. AI 활용 내역 및 참고 자료

1. 과제 개요 및 목적

이번 과제에서는 Python과 Gymnasium을 이용하여 FrozenLake 환경에서 Q-Learning 알고리즘을 구현했다. FrozenLake는 4×4 격자에서 구멍을 피해 목표 지점까지 이동하는 환경으로, 에이전트가 행동에 따른 보상을 받으면서 목표에 도달하는 방법을 학습하도록 했다.

학습에는 상태 16개와 행동 4개로 구성된 Q-table을 사용했다. ε-greedy 방식을 통해 무작위 행동을 선택하는 탐험과 Q값이 높은 행동을 선택하는 활용을 함께 진행했다. 이후 행동의 결과로 얻은 보상과 다음 상태의 Q값을 이용하여 기존 Q값을 갱신하도록 했다.

총 5,000번의 Episode 동안 학습을 진행하고, 학습이 끝난 후에는 별도로 200번의 평가를 실시했다. 평가 과정에서는 성공률과 목표까지 이동한 경로 길이를 확인했다. 또한 학습 성공률 및 평균 보상 그래프, Q-table 히트맵, 정책 격자를 생성하여 학습 결과를 분석했다.

2. FrozenLake Q-Learning 구현

2.1 FrozenLake 환경 및 Q-table 구성

먼저 강화학습에 필요한 라이브러리를 불러오고 FrozenLake 환경을 설정했다.

코드 1~11줄

```
import gymnasium as gym
import numpy as np
import matplotlib.pyplot as plt


# FrozenLake 4x4 환경 생성 (미끄러짐 없음)
env = gym.make("FrozenLake-v1",
               map_name="4x4", is_slippery=False)

# 랜덤 시드 설정
rng = np.random.default_rng(0)
```

Gymnasium은 FrozenLake 환경을 실행하기 위해 사용했고, NumPy는 Q-table 생성과 계산에 사용했다. Matplotlib은 학습 결과를 그래프로 나타내기 위해 불러왔다.

FrozenLake는 4×4 크기의 맵으로 설정했으며, is_slippery=False를 적용하여 에이전트가 선택한 방향대로 이동하도록 했다. 또한 난수 생성기의 시드를 0으로 설정하여 무작위 행동 선택 과정을 재현할 수 있도록 했다.

FrozenLake 환경을 만든 후에는 각 상태와 행동의 가치를 저장할 Q-table을 생성했다.

코드 13~21줄

```
# 상태 16개, 행동 4개의 Q-table 생성 및 초기화
Q = np.zeros((env.observation_space.n,
              env.action_space.n))

# 학습률과 할인율 설정
alpha, gamma = 0.1, 0.95

# Episode별 누적 보상 저장
returns = []
```

FrozenLake는 16개의 상태와 4개의 행동으로 구성되어 있으므로 Q-table을 16×4 크기로 생성했다. 처음에는 각 행동의 가치를 알 수 없기 때문에 모든 Q값을 0으로 초기화했다.

학습률(α)은 0.1, 할인율(γ)은 0.95로 설정했다. 학습률은 새로운 경험을 Q값에 얼마나 반영할지 결정하고, 할인율은 미래에 받을 보상을 얼마나 중요하게 생각할지 결정한다. 또한 returns 리스트를 만들어 Episode마다 얻은 누적 보상을 저장하도록 했다.

2.2 ε-greedy 행동 선택

코드 24~36줄

```
# ε-greedy 방식으로 행동 선택
def choose_action(state, epsilon, rng=rng):

    # 확률 epsilon으로 무작위 행동 선택 (탐험)
    if rng.random() < epsilon:
        return int(rng.integers(4))

    # 현재 상태에서 Q 값이 가장 큰 행동 탐색
    best = np.flatnonzero(
        Q[state] == Q[state].max())

    # Q 값이 가장 큰 행동 중 무작위 선택 (활용)
    return int(rng.choice(best))
```

에이전트가 행동을 선택할 때는 ε-greedy 방식을 사용했다. 이 방식은 무작위 행동을 선택하는 탐험과 현재 Q값을 기준으로 행동을 선택하는 활용을 함께 사용하는 방법이다.

난수가 ε보다 작으면 4개의 행동 중 하나를 무작위로 선택하고, 그렇지 않으면 현재 상태에서 Q값이 가장 큰 행동을 선택하도록 했다. Q값이 같은 행동이 여러 개일 경우에는 그중 하나를 무작위로 선택하도록 구성했다.

2.3 Q-Learning 학습 및 평가 과정

코드 39~72줄 — Q-Learning 학습

```
# 총 5,000번의 Episode 동안 학습
for episode in range(5000):

    # 환경 초기화 및 현재 상태 가져오기
    state, _ = env.reset(seed=0 if episode == 0 else None)

    # Episode가 진행될수록 탐험 확률 감소
    epsilon = max(0.05, 0.999 ** episode)
    total = 0

    # Episode가 종료될 때까지 행동 반복
    while True:

        # ε-greedy 방식으로 행동 선택
        action = choose_action(state, epsilon)

        # 행동 실행 후 다음 상태, 보상, 종료 여부 확인
        ns, r, terminated, truncated, _ = env.step(action)

        # Q-Learning의 TD target 계산
        target = r if terminated else r + gamma * Q[ns].max()
```

```
        # Q 값 갱신
        Q[state, action] += alpha * (target - Q[state, action])

        # 보상 누적 및 현재 상태 갱신
        total += r
        state = ns

        # 목표 도달, 구멍 진입 또는 시간 제한 시 종료
        if terminated or truncated:
            break

    # Episode별 누적 보상 저장
    returns.append(total)
```

Q-Learning 학습은 총 5,000번의 Episode 동안 진행했다. 각 Episode가 시작되면 환경을 초기화하고, 학습이 진행될수록 ε값이 감소하도록 설정했다. 이를 통해 초기에는 다양한 행동을 시도하고, 학습이 진행되면서 Q값이 높은 행동을 선택하는 비율이 증가하도록 했다.

에이전트가 행동을 실행하면 다음 상태와 보상을 받아 Q값을 갱신한다. 이때 목표 지점에 도달하거나 구멍에 빠져 종료된 경우에는 실제로 받은 보상만 사용하고, 그렇지 않은 경우에는 다음 상태에서 얻을 수 있는 최대 Q값에 할인율을 적용하여 계산했다.

또한 terminated와 truncated를 구분하여 Q값을 계산하되, 두 경우 모두 Episode의 반복은 종료하도록 했다. 마지막으로 Episode마다 얻은 누적 보상을 returns 리스트에 저장했다.

코드 76~92줄 — Q-table 저장 및 평가 환경 설정

```
# 학습된 Q-table 파일 저장
np.save("q_table.npy", Q)


# 저장된 Q-table 불러오기
Q = np.load("q_table.npy")

# 평가용 FrozenLake 환경 생성
eval_env = gym.make("FrozenLake-v1",
                    map_name="4x4", is_slippery=False)

# 평가용 랜덤 시드 설정
eval_rng = np.random.default_rng(1000)

# 성공 횟수 및 성공 경로 길이 저장
wins = 0
path_lengths = []
```

5,000번의 Episode 동안 학습한 Q-table을 q_table.npy 파일로 저장하고, 저장된 파일을 다시 불러오도록 했다. 이를 통해 학습 결과를 파일로 보관하고 평가 과정에서 사용할 수 있도록 했다.

평가에는 학습과 동일한 조건의 FrozenLake 환경을 새로 생성했다. 또한 성공 횟수를 저장할 wins 변수와 목표 지점까지 이동한 경로 길이를 기록할 path_lengths 리스트를 준비했다.

코드 94~116줄 — 학습된 Q-table 평가

```
# 학습된 Q-table로 총 200번 평가
for ep in range(200):

    # 평가 환경 초기화
    state, _ = eval_env.reset(seed=1000 if ep == 0 else None)
    steps = 0

    # 탐험 없이 목표 지점까지 이동
    while True:

        # Q 값이 가장 큰 행동 선택
        action = choose_action(state, 0.0, eval_rng)

        # 행동 실행 후 다음 상태와 보상 확인
        state, r, term, trunc, _ = eval_env.step(action)
        steps += 1

        # Episode 종료 시 성공 여부 확인
        if term or trunc:
            if r == 1:
                wins += 1
                path_lengths.append(steps)
            break
```

학습된 Q-table의 성능을 확인하기 위해 총 200번의 Episode를 평가했다. 학습 과정과 달리 ε값을 0으로 설정하여 무작위 탐험 없이 Q값이 가장 높은 행동만 선택하도록 했다.

각 Episode에서는 에이전트가 이동한 횟수를 steps에 기록했다. 목표 지점에 도달하여 보상 1을 받은 경우에는 성공 횟수를 증가시키고, 해당 Episode의 이동 횟수를 리스트에 저장했다. 이를 통해 성공률뿐만 아니라 목표까지 얼마나 짧은 경로로 이동했는지도 확인할 수 있도록 했다.

코드 119~131줄 — 평가 결과 출력

```
# 최종 평가 결과 출력
print(f"Evaluation Success Rate: {wins / 200 * 100:.2f}%")
print(f"Successful Episodes: {wins}/200")

# 성공한 Episode의 경로 길이 확인
if path_lengths:
    print(f"Average Path Length: {np.mean(path_lengths):.2f}")
    print(f"Min Path Length: {np.min(path_lengths)}")
    print(f"Max Path Length: {np.max(path_lengths)}")

# 학습 및 평가 환경 종료
eval_env.close()
env.close()
```

평가가 끝난 후에는 전체 200번의 Episode 중 성공한 비율과 성공 횟수를 출력하도록 했다. 또한 성공한 Episode들의 평균 이동 횟수와 최소·최대 이동 횟수를 계산하여 출력했다.

실행 결과 200번의 평가에서 모두 목표 지점에 도달하여 성공률 100%를 기록했으며, 평균·최소·최대 이동 횟수는 모두 6회로 나타났다. 마지막으로 학습 환경과 평가 환경을 종료하도록 했다.

2.4 학습 결과 시각화

① 학습 성공률 및 평균 보상 그래프

코드 133~156줄

```
# 1. 학습 성공률 및 평균 보상 그래프

# 500 Episode마다 평균 성공률 및 평균 보상 계산
episodes = []
success_rates = []

for i in range(0, len(returns), 500):
    episodes.append(i + 500)
    success_rates.append(np.mean(returns[i:i + 500]))

# 학습 성공률 및 평균 보상 그래프 생성
plt.figure(figsize=(9, 5))
plt.plot(episodes, success_rates, marker="o")

# 그래프 제목 및 축 설정
plt.title("Training Success Rate and Average Reward")
plt.xlabel("Episode")
plt.ylabel("Success Rate / Average Reward")
plt.ylim(0, 1)
plt.grid(True)
```

```
# 그래프 이미지 저장
plt.savefig("training_success_rate.png", dpi=300,
            bbox_inches="tight")
```

학습이 진행되면서 성공률이 어떻게 변하는지 확인하기 위해 Episode별 누적 보상을 이용하여 그래프를 생성했다. 500번의 Episode마다 평균값을 계산하고, 이를 선 그래프로 나타내도록 했다.

FrozenLake에서는 목표 지점에 도달하면 보상 1을 받고, 실패하면 0을 받기 때문에 평균 보상과 성공률이 같은 값으로 나타난다. 그래프의 가로축은 Episode 수, 세로축은 성공률 및 평균 보상으로 설정했으며, 결과를 training_success_rate.png 파일로 저장했다.

② Q-table 히트맵

코드 158~184줄

```
# 2. Q-table 히트맵

# 학습된 Q-table 히트맵 생성
plt.figure(figsize=(8, 9))
plt.imshow(Q, cmap="YlGnBu", aspect="auto")

# 그래프 제목 및 축 설정
plt.title("FrozenLake Q-table Heatmap")
plt.xlabel("Action")
plt.ylabel("State")

# 행동 번호와 상태 번호 표시
plt.xticks(range(4), ["Left", "Down", "Right", "Up"])
plt.yticks(range(16))

# 각 상태와 행동의 Q값 표시
for s in range(16):
    for a in range(4):
        plt.text(a, s, f"{Q[s, a]:.2f}",
                 ha="center", va="center", fontsize=9)
```

```
# Q값에 따른 색상 표시
plt.colorbar(label="Q-value")
plt.tight_layout()

# 히트맵 이미지 저장
plt.savefig("q_table_heatmap.png", dpi=300)
```

학습된 Q-table의 값을 확인하기 위해 히트맵을 생성했다. 세로축에는 16개의 상태를, 가로축에는 왼쪽·아래·오른쪽·위쪽의 4개 행동을 표시했다.

각 칸에는 해당 상태에서 특정 행동을 선택했을 때의 Q값을 소수점 둘째 자리까지 나타냈다. 또한 Q값의 크기에 따라 색상이 달라지도록 설정하여 어떤 상태와 행동의 가치가 높은지 쉽게 비교할 수 있도록 했다. 생성된 히트맵은 q_table_heatmap.png 파일로 저장했다.

③ 정책 격자(Policy Grid)

코드 186~227줄 — 정책 격자 생성 및 행동 방향 표시

```
# 3. 정책 격자 (Policy Grid)

# FrozenLake 4x4 맵 구성
lake_map = [
    "SFFF",
    "FHFH",
    "FFFH",
    "HFFG"
]

# 행동 번호에 따른 이동 방향
directions = ["←", "↓", "→", "↑"]

# 정책 격자 생성
fig, ax = plt.subplots(figsize=(6, 6))

for row in range(4):
    for col in range(4):

        # 행과 열을 상태 번호로 변환
        state = row * 4 + col
```

```
        tile = lake_map[row][col]

        # 구멍과 목표 지점은 별도로 표시
        if tile == "H" or tile == "G":
            symbol = tile

        # Q 값이 모두 0이면 학습되지 않은 상태로 표시
        elif np.allclose(Q[state], 0):
            symbol = "?"

        # Q 값이 가장 큰 행동을 방향으로 표시
        else:
            action = np.argmax(Q[state])
            symbol = directions[action]

        # 시작 위치 표시
        if tile == "S":
            symbol = "S\n" + symbol

        ax.text(col, row, symbol,
                ha="center", va="center", fontsize=20)
```

학습된 Q-table을 이용하여 에이전트가 각 위치에서 어떤 방향으로 이동하는지 확인할 수 있는 정책 격자를 만들었다. FrozenLake의 4×4 맵을 구성하고, 각 위치에서 Q값이 가장 큰 행동을 화살표로 표시하도록 했다.

시작 지점은 S, 구멍은 H, 목표 지점은 G로 표시했다. 또한 모든 Q값이 0인 상태는 물음표로 표시하여 학습된 행동 방향이 정해지지 않은 상태와 구분했다.

코드 229~245줄 — 격자 설정 및 이미지 저장

```
# 격자 눈금 및 제목 설정
ax.set_xticks(np.arange(-0.5, 4, 1), minor=True)
ax.set_yticks(np.arange(-0.5, 4, 1), minor=True)
ax.grid(which="minor", color="gray", linewidth=1)

ax.set_xlim(-0.5, 3.5)
ax.set_ylim(3.5, -0.5)
ax.set_xticks([])
ax.set_yticks([])
ax.set_title("FrozenLake Greedy Policy")

plt.tight_layout()

# 정책 격자 이미지 저장
plt.savefig("policy_grid.png", dpi=300,
            bbox_inches="tight")
plt.show()
```

각 상태의 위치를 구분하기 쉽도록 격자선을 추가하고, FrozenLake 환경과 동일한 4×4 형태로 표시했다. 가로축과 세로축의 눈금은 숨기고 제목을 설정하여 이동 방향이 잘 보이도록 했다.

완성된 정책 격자는 policy_grid.png 파일로 저장했으며, 마지막에 plt.show()를 사용하여 학습 성공률 그래프, Q-table 히트맵, 정책 격자를 화면에 표시하도록 했다.

3. 실행 결과 및 분석

3.1 학습 성공률 및 평균 보상

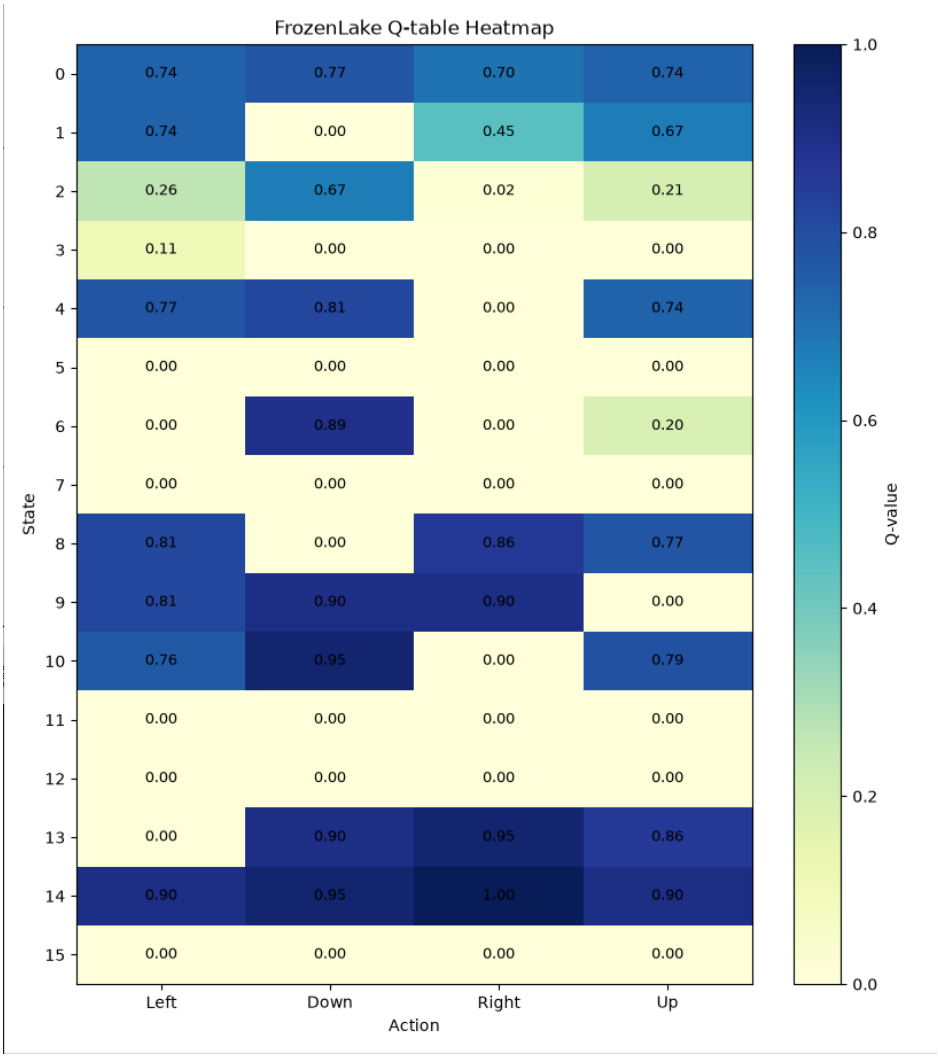


학습 성공률 및 평균 보상 그래프를 확인한 결과, Episode가 증가할수록 성공률이 높아지는 모습을 볼 수 있었다. 초기 500 Episode에서는 성공률이 약 0.13이었지만, 1,000 Episode에서는 약 0.46, 2,000 Episode에서는 약 0.84까지 증가했다.

이후 3,000 Episode부터는 성공률이 약 0.93 수준에 도달했고, 4,000~5,000 Episode 구간에서는 약 0.93~0.94로 유지되었다. 학습 과정에서 탐험 확률 ε는 점차 감소하며, 약 2,994 Episode부터 최솟값인 0.05로 유지된다. 이에 따라 3,000 Episode 이후에는 탐험 비율이 일정해지고 Q값도 점차 안정되면서 성공률이 비슷한 수준으로 유지된 것으로 볼 수 있다.

학습 과정에서는 ε-greedy 방식으로 무작위 탐험을 계속하기 때문에 성공률이 100%에 도달하지는 않았다. 반면 학습이 끝난 후 탐험 없이 진행한 200번의 평가에서는 모두 성공하여 최종 성공률 100%를 기록했다.

3.2 Q-table 히트맵 및 정책 격자

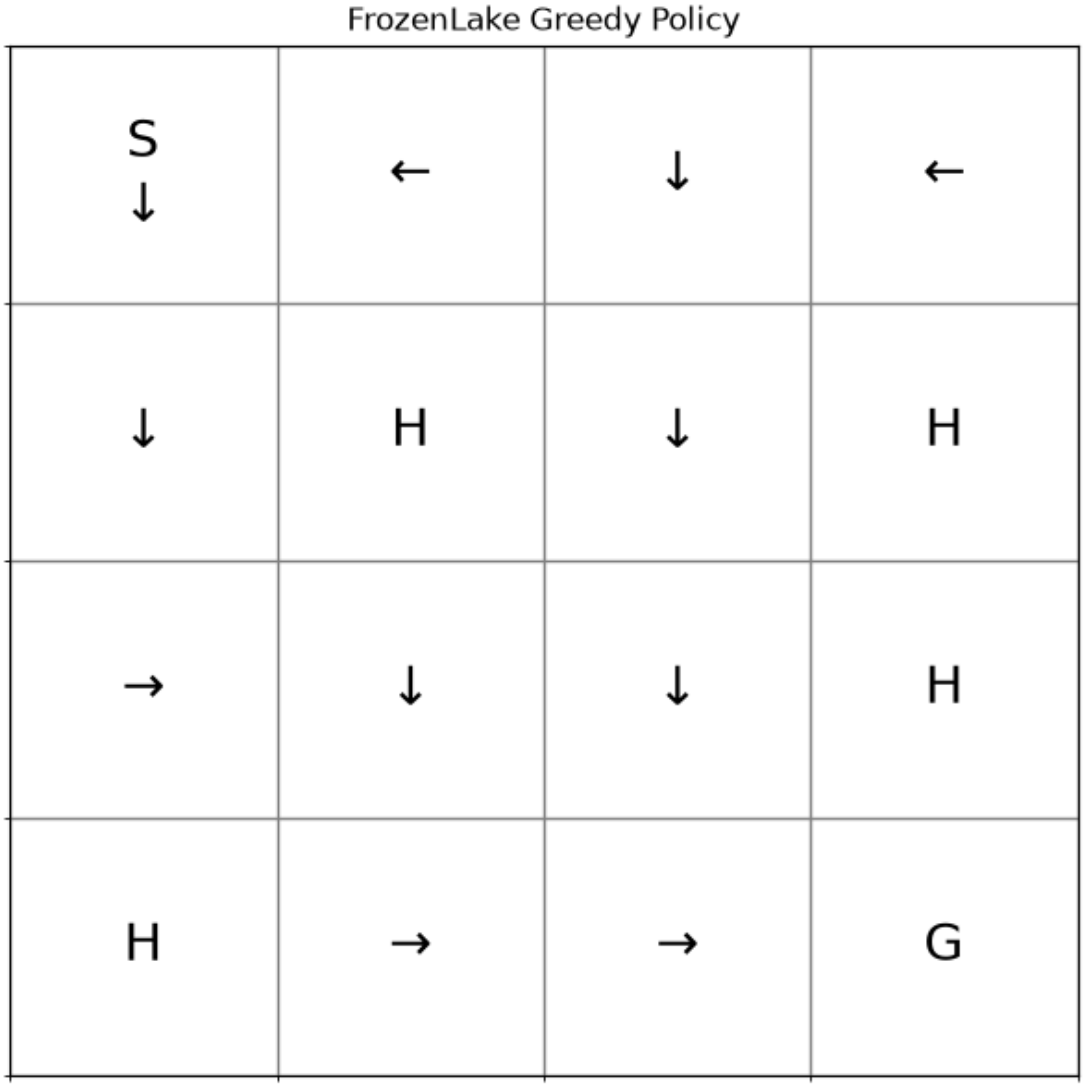


Q-table 히트맵에서는 16개의 상태에서 각 행동이 가지는 Q값을 확인할 수 있었다. 목표 지점에 가까워질수록 올바른 이동 방향의 Q값이 높게 나타나는 것을 볼 수 있었다.

특히 상태 14에서 오른쪽으로 이동하는 행동의 Q값은 1.00으로 가장 높게 나타났다. 상태 14에서 오른쪽으로 이동하면 바로 목표 지점에 도착하여 보상 1을 받을 수 있기 때문이다. 반면 구멍에 해당하는 상태 5, 7, 11, 12와 목표 지점인 상태 15에서는 Q값이 모두 0으로 나타났다.

이 상태들은 Episode가 종료되는 상태이므로 추가 행동을 수행하지 않아 Q값의 초기값인 0이 그대로 유지된 것이다. 구멍의 위험은 구멍으로 이동하는 행동의 Q값에 반영되며, 실제로 상태 4에서 오른쪽으로 이동하거나 상태 8에서 아래쪽으로 이동하는 행동의 Q값이 0.00으로 나타났다.

이를 통해 Q-Learning이 행동에 따른 보상을 바탕으로 Q값을 갱신하고, 목표 지점에 도달하는 데 유리한 행동의 가치를 높여 나간다는 것을 확인할 수 있었다.



정책 격자는 학습된 Q-table에서 각 상태의 Q값이 가장 높은 행동을 화살표로 나타낸 결과이다. 시작 지점 S에서 아래쪽으로 이동한 후 구멍을 피해 목표 지점 G까지 도달하는 경로가 형성된 것을 확인할 수 있었다.

대표적인 이동 경로는 상태 0 → 4 → 8 → 9 → 13 → 14 → 15이며, 총 6번의 이동으로 목표 지점에 도착한다. 이는 해당 맵에서 목표까지 도달할 수 있는 최단 이동 횟수와 같다. 평가 결과에서도 200번 모두 성공했으며, 평균 이동 횟수와 최소·최대 이동 횟수가 모두 6회로 나타났다.

다만 모든 상태가 동일한 수준으로 학습된 것은 아니었다. 상태 1에서는 오른쪽보다 왼쪽 행동의 Q값이 높게 나타났는데, 이는 학습 과정에서 아래쪽 경로를 주로 이용하면서 일부 상태의 Q값이 충분히 학습되지 않았기 때문으로 볼 수 있다. 따라서 학습된 정책은 시작 지점에서 최단 경로로 이동할 수 있었지만, 모든 상태에서 최적의 행동을 학습했다고 보기는 어렵다.

3.3 강화학습 핵심 개념 및 Q값 갱신 과정

강화학습에서 MDP(Markov Decision Process)는 상태, 행동, 상태 전이 확률, 보상, 할인율로 구성된다. 이번 FrozenLake 환경에서는 16개의 상태와 4개의 행동을 사용했으며, is_slippery=False로 설정하여 선택한 방향에 따라 이동하도록 했다. 목표 지점에 도착하면 보상 1을 받고, 나머지 경우에는 보상 0을 받는다. 할인율은 0.95로 설정했다.

Return은 현재 시점부터 미래에 받을 보상을 할인율을 적용하여 더한 값이다. 예를 들어 목표 지점에 도달하기까지 6번 이동하고 마지막에 보상 1을 받는다면, 시작 상태에서의 Return은 다음과 같이 계산할 수 있다.

$G = 0 + 0.95 \times 0 + 0.95^2 \times 0 + \ldots + 0.95^5 \times 1 \approx 0.774$

실제로 Q-table 히트맵에서 시작 상태인 0에서 아래쪽으로 이동하는 행동의 Q값이 약 0.77로 나타나 이 계산과 비슷한 값을 확인할 수 있었다.

Q-Learning에서는 현재 Q값과 새롭게 계산한 목표값의 차이를 이용하여 Q값을 갱신한다. 예를 들어 학습률이 0.1, 할인율이 0.95이고 현재 Q값이 0.80이라고 가정하자. 즉시 보상은 0이고 다음 상태의 최대 Q값이 1.00이라면 다음과 같이 계산된다.

$\text{TD target} = 0 + 0.95 \times 1.00 = 0.95$

$\text{새로운 Q값} = 0.80 + 0.1 \times (0.95 - 0.80) = 0.815$

이처럼 기존 Q값을 한 번에 바꾸지 않고, 새로운 경험을 일정 비율만큼 반영하여 갱신한다. 위 수치 예제는 Q-Learning의 계산 과정을 설명하기 위한 가정이며, 실제 코드에서도 같은 수식을 사용해 학습을 진행했다.

4. 결론 및 고찰

이번 과제에서는 Gymnasium의 FrozenLake 환경에서 Q-Learning 알고리즘을 구현하고, 학습 과정과 결과를 확인했다. 16개의 상태와 4개의 행동으로 구성된 Q-table을 생성하고, ε-greedy 방식을 이용하여 탐험과 활용을 반복하면서 Q값을 갱신하도록 했다.

총 5,000번의 Episode 동안 학습을 진행한 결과, 초기에는 낮았던 성공률이 점차 증가하여 학습 후반에는 약 93~94% 수준으로 유지되었다. 학습이 끝난 후 무작위 탐험 없이 200번의 평가를 진행했을 때는 모두 목표 지점에 도달하여 성공률 100%를 기록했다. 또한 평균·최소·최대 이동 횟수가 모두 6회로 나타나 최단 경로로 이동한다는 것도 확인할 수 있었다.

Q-table 히트맵과 정책 격자를 통해 학습 결과를 시각적으로 확인할 수 있었다. 특히 목표 지점에 가까워질수록 올바른 행동의 Q값이 높게 나타났으며, 정책 격자에서도 구멍을 피해 목표 지점까지 이어지는 이동 경로를 확인할 수 있었다. 다만 일부 상태에서는 Q값이 충분히 학습되지 않은 모습도 나타났다. 이를 통해 Q-Learning이 보상을 바탕으로 행동의 가치를 학습하지만, 상태별 방문 횟수에 따라 학습 정도에 차이가 발생할 수 있다는 것을 확인했다.

이번 과제에서는 is_slippery=False로 설정하여 에이전트가 선택한 방향대로 이동하는 환경에서 학습을 진행했다. 평가에서도 무작위 탐험을 제외했기 때문에 동일한 조건에서 안정적인 결과를 얻을 수 있었다. 다만 이동 방향에 불확실성이 없는 환경에서 얻은 결과이므로, 다른 환경에서도 동일한 성능이 나타난다고 보기는 어렵다. 이후에는 is_slippery=True로 설정하거나 학습률과 탐험 확률을 변경하여 학습 결과가 어떻게 달라지는지 비교해 보고 싶다.

5. AI 활용 내역 및 참고 자료

5.1 AI 활용 내역

이번 과제에서는 ROBIT 강화학습 2일차 강의자료를 학습하면서 이해하기 어려웠던 개념을 정리하고, 실행 결과를 분석하는 과정에서 ChatGPT를 활용했다.

먼저 강의자료에 등장하는 MDP와 Return의 개념을 이해하는 데 도움을 받았다. 특히 상태, 행동, 보상, 할인율이 각각 어떤 역할을 하는지 FrozenLake 환경에 적용하여 확인했으며, 미래에 받을 보상에 할인율을 적용하는 이유와 Return의 계산 과정을 예시를 통해 정리했다.

또한 Q-Learning의 핵심인 TD target과 Q값 갱신 수식을 이해하는 데 활용했다. 처음에는 현재 Q값과 다음 상태의 최대 Q값을 이용하여 새로운 Q값을 계산하는 과정이 어려웠지만, 학습률과 할인율을 적용한 수치 예제를 통해 계산 순서를 확인할 수 있었다.

ε-greedy 방식에서 탐험과 활용이 어떻게 구분되는지, 학습이 진행되면서 탐험 확률을 감소시키는 이유도 함께 정리했다.

추가로 학습 결과를 확인하기 위한 성공률 그래프, Q-table 히트맵, 정책 격자 등의 시각화 코드를 작성하는 데 도움을 받았다. 실행 후에는 학습 성공률과 최종 평가 성공률에 차이가 발생하는 이유를 확인하고, 히트맵의 Q값과 정책 격자에 표시된 이동 방향을 해석하는 데 활용했다. 마지막으로 코드의 주요 기능과 실행 결과를 보고서로 정리하는 과정에서도 도움을 받았다.

5.2 참고 자료

1. ROBIT,「MDP와 Q-Learning 강화학습 2일차」, 2026.10.29.
2. RangDev,「[Deep RL Course] Q-Learning 실습(1) - Frozen Lake」, 2025.01.08. https://ysl-iras.tistory.com/entry/Deep-RL-Course-Q-Learning-실습1-Frozen-Lake
3. 잡덕의 코딩,「[RL] gymnasium frozen lake 강화 학습 - 2」, 2025.02.03. https://yhj9855.com/entry/RL-gymnasium-frozen-lake-강화-학습-2
4. Gymnasium Documentation,「Solving Frozenlake with Tabular Q-Learning」. https://gymnasium.farama.org/tutorials/training_agents/frozenlake_q_learning/
5. OpenAI, ChatGPT — 강화학습 개념 이해, 수식 계산, 시각화 코드 작성 및 실행 결과 분석 보조.